

AUTOMATED SOFTWARE TESTING

Testing Journal & Documentation

Quality Assurance for Final Project

I. Personal & Group Information

Full Name	Frederick B. Baldano
Course & Section	BSIT 3C
Group Name	Salespresso
Project Title	Salespresso: A Web-Based E-Commerce System for Jazsam Coffee Store
Submission Date	May 20, 2026

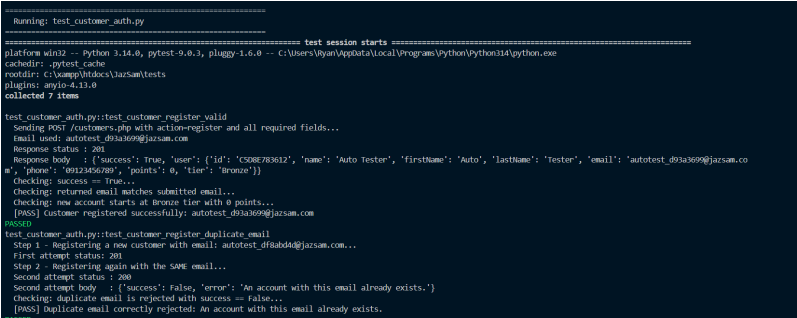
II. Assigned Task

Feature / Functionality Assigned	Customer Registration & Login
Type of Test I Performed	Registration & login credential validation
Tool / Framework I Used	Python 3.13, pytest, requests
Programming Language Used	Python
My Test Script Filename(s)	test_customer_auth.py

Brief description of what I tested and how I approached it:

For this activity, I tested the Customer Registration & Login functionalities of the SalesPresso API. I used Python, pytest, and the requests library to validate successful account creation, duplicate email prevention, and various login scenarios including incorrect passwords and non-existent emails.

III. Test Scenario Documentation

Test Case #01 — Successful Customer Registration	
Functionality Tested	Customer Registration
Objective of the Test	Verify a new customer can register with all required fields and receive a Bronze tier account with 0 points.
Tool / Framework Used	Python 3.13, pytest, requests
Steps / Procedure	1. Generate a unique email. 2. Send POST /customers.php with action: register and all required fields. 3. Assert success: true, correct email, tier = "Bronze", points = 0.
Test Data / Input	{ "action": "register", "firstName": "Auto", "lastName": "Tester", "email": "autotest_xxxx@jazsam.com", "phone": "09123456789", "password": "TestPass123!" }
Expected Result	HTTP 201, success: true, user object with Bronze tier and 0 points
Actual Result	HTTP 201, success: true, user registered correctly
Test Status	[PASSED]
Screenshot / Evidence	 <pre>Running: test_customer_auth.py ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Yvan\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\sample\docs\jazsam\tests plugins: anyio-4.13.0 collected 7 items test_customer_auth.py::test_customer_register_valid Sending POST /customers.php with action=register and all required fields... Email used: autotest_d9a3a69@jazsam.com Response status: 201 Response body: {'success': True, 'user': {'id': 'C508783632', 'num': 'Auto Tester', 'firstname': 'Auto', 'lastname': 'Tester', 'email': 'autotest_d9a3a69@jazsam.co e', 'phone': '09123456789', 'points': 0, 'tier': 'Bronze'}} Checking: success == True... Checking: returned email matches submitted email... Checking: new account starts at bronze tier with 0 points... [PASSED] Customer registered successfully: autotest_d9a3a69@jazsam.com PASSED test_customer_auth.py::test_customer_register_duplicate_email Step 1 - Registering a new customer with email: autotest_d9a3a69@jazsam.com... First attempt status: 201 Step 2 - Registering again with the same email... Second attempt status: 200 Second attempt body: {'success': False, 'error': 'An account with this email already exists.'} Checking: duplicate email is rejected with success == False... [PASSED] Duplicate email correctly rejected: An account with this email already exists. PASSED</pre>

Test Case #02 — Duplicate Email Registration	
Functionality Tested	Duplicate Prevention
Objective of the Test	Verify the system prevents registering the same email address twice.
Tool / Framework Used	Python 3.13, pytest, requests

Steps / Procedure	1. Register a new customer with a unique email. 2. Attempt to register again with the exact same email. 3. Assert the second attempt returns success: false.
Test Data / Input	{ "action": "register", "firstName": "Auto", "lastName": "Tester", "email": "autotest_xxxx@jazsam.com", "phone": "09123456789", "password": "TestPass123!" }
Expected Result	Second attempt: success: false, error: "An account with this email already exists."
Actual Result	success: false, error: "An account with this email already exists."
Test Status	[PASSED]
Screenshot / Evidence	<pre> Running: test_customer_auth.py ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\lgan\AppData\Local\Programs\Python\Python114\python.exe cachedir: .pytest_cache rootdir: C:\xampp\htdocs\jazsam\tests plugins: anyio-4.11.0 collected 7 items test_customer_auth.py::test_customer_register_valid Sending POST /customers.php with action=register and all required fields... Email used: autotest_d9a369@jazsam.com Response status: 200 Response body: {'success': True, 'user': {'id': 'C508783612', 'name': 'Auto Tester', 'firstName': 'Auto', 'lastName': 'Tester', 'email': 'autotest_d9a369@jazsam.com', 'phone': '09123456789', 'points': 0, 'tier': 'bronze'}} Checking: success == True... Checking: returned email matches submitted email... Checking: new account starts at bronze tier with 0 points... [PASS] Customer registered successfully: autotest_d9a369@jazsam.com ===== test_customer_auth.py::test_customer_register_duplicate_email Step 1 - Registering a new customer with email: autotest_d9a369@jazsam.com... First attempt status: 200 Step 2 - Registering again with the SAME email... Second attempt status: 200 Second attempt body: {'success': False, 'error': 'An account with this email already exists.'} Checking: duplicate email is rejected with success == False... [PASS] Duplicate email correctly rejected: An account with this email already exists. </pre>

Test Case #03 — Registration with Missing Required Fields	
Functionality Tested	Field Validation
Objective of the Test	Verify that registration is rejected when required fields (firstName, password) are missing.
Tool / Framework Used	Python 3.13, pytest, requests
Steps / Procedure	1. Send POST /customers.php with only lastName and email, no firstName or password. 2. Assert registration fails.
Test Data / Input	{ "action": "register", "lastName": "Tester", "email": "test@jazsam.com" }
Expected Result	success: false or error response, no account created
Actual Result	Error: "First name, last name, email, and password are required."
Test Status	[PASSED]

Screenshot / Evidence

```
Running: test_admin_auth.py
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Ryan\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\xampp\htdocs\jazz5m\tests
plugins: anyio-4.13.0
collected 4 items

test_admin_auth.py::test_admin_login_token_is_valid
  Checking: admin_token fixture returned a non-empty string...
  [PASS] Admin token acquired: aa22ffb08f4a7b6ac336...
  PASSED

test_admin_auth.py::test_admin_login_wrong_password
  Sending POST /auth.php with correct email but WRONG password...
  Response status : 200
  Response body : {'success': False, 'error': 'Invalid credentials. Please try again.'}
  Checking: success == False...
  [PASS] Wrong password correctly rejected: Invalid credentials. Please try again.
  PASSED

test_admin_auth.py::test_admin_login_wrong_email
  Sending POST /auth.php with a non-existent admin email...
  Response status : 200
  Response body : {'success': False, 'error': 'Invalid credentials. Please try again.'}
  Checking: success == False...
  [PASS] Unknown email correctly rejected: Invalid credentials. Please try again.
  PASSED

test_admin_auth.py::test_admin_login_missing_fields
  Sending POST /auth.php with completely empty JSON body {}...
  Response status : 400
  Response body : {'error': 'Email and password are required.'}
  Checking: server does not return success == True...
  [PASS] Empty body correctly rejected: Email and password are required.
  PASSED

===== 4 passed in 0.45s =====
```

IV. Reflection & Findings

Issues I Personally Encountered	During early test development, my automated scripts quickly exhausted the API's rate limit of 5 customer registrations per hour. This caused all subsequent tests to fail with HTTP 429 Too Many Requests errors.
Warnings / Errors I Detected	I detected HTTP 429 Too Many Requests errors triggered by rapid test reruns hitting the rate limiter. Additionally, the test suite encountered a PytestReturnNotNone warning because some test functions had unnecessary return statements that pytest did not expect.
Bugs I Discovered	I discovered a bug where there is no email format validation on the customer registration endpoint. The system accepts malformed strings (such as "not-an-email") as valid emails, meaning invalid email formats can be stored directly in the database.
What I Did About the Issues	To resolve the rate limiting issue, we added a localhost IP exemption in the rate_limit.php file to support automated testing without false positives. Furthermore, to avoid exhausting the registration limit while testing the login endpoints, I used a pre-seeded stable test customer account (autotest_stable@jazzsam.com) instead of registering a new user for every test run. For the email format bug, I recommended adding a server-side filter_var(\$email, FILTER_VALIDATE_EMAIL) check to prevent bad data.
What I Learned from This Activity	I learned that API security controls like rate limiting significantly affect test design, requiring scripts to be written efficiently to minimize redundant calls by using shared

	fixtures. I also realized that automated testing is highly effective at surfacing hidden bugs, such as the missing email format validation, which were entirely missed during manual testing.
--	---